We then simplified it to the law that mapping the identity function over a computation should have no effect:

```
map(y)(x => x) == y
```

Can we express this? Not exactly. This property implicitly states that the equality holds *for all* choices of `y`, for all types. We're forced to pick particular values for `y`:

```
val pint = Gen.choose(0,10) map (Par.unit(_))
val p4 =
  forAllPar(pint)(n => equal(Par.map(n)(y => y), n))
```

We can certainly range over more choices of `y`, but what we have here is probably good enough. The implementation of `map` can't care about the values of our parallel computation, so there isn't much point in constructing the same test for `Double`, `String`, and so on. What *can* affect `map` is the *structure* of the parallel computation. If we wanted greater assurance that our property held, we could provide richer generators for the structure. Here, we're only supplying `Par` expressions with one level of nesting.

EXERCISE 8.16

Hard: Write a richer generator for `Par[Int]`, which builds more deeply nested parallel computations than the simple ones we gave previously.

EXERCISE 8.17

Express the property about `fork` from chapter 7, that `fork(x) == x`.

8.5 Testing higher-order functions and future directions

So far, our library seems quite expressive, but there's one area where it's lacking: we don't currently have a good way to test higher-order functions. While we have lots of ways of generating *data* using our generators, we don't really have a good way of generating *functions*.

For instance, let's consider the `takeWhile` function defined for `List` and `Stream`. Recall that this function returns the longest prefix of its input whose elements all satisfy a predicate. For instance, `List(1,2,3).takeWhile(_ < 3)` results in `List(1,2)`. A simple property we'd like to check is that for any list, `s: List[A]`, and any `f: A => Boolean`, the expression `s.takeWhile(f).forall(f)` evaluates to `true`. That is, every element in the returned list satisfies the predicate.[11]

[11] In the Scala standard library, `forall` is a method on `List` and `Stream` with the signature `def forall[A](f: A => Boolean): Boolean`.

EXERCISE 8.18

Come up with some other properties that `takeWhile` should satisfy. Can you think of a good property expressing the relationship between `takeWhile` and `dropWhile`?

We could certainly take the approach of only examining *particular* arguments when testing higher-order functions. For instance, here's a more specific property for `takeWhile`:

```
val isEven = (i: Int) => i%2 == 0
val takeWhileProp =
  Prop.forAll(Gen.listOf(int))(ns => ns.takeWhile(isEven).forall(isEven))
```

This works, but is there a way we could let the testing framework handle generating functions to use with `takeWhile`?[12] Let's consider our options. To make this concrete, let's suppose we have a `Gen[Int]` and would like to produce a `Gen[String => Int]`. What are some ways we could do that? Well, we could produce `String => Int` functions that simply ignore their input string and delegate to the underlying `Gen[Int]`:

```
def genStringIntFn(g: Gen[Int]): Gen[String => Int] =
  g map (i => (s => i))
```

This approach isn't sufficient though. We're simply generating *constant* functions that ignore their input. In the case of `takeWhile`, where we need a function that returns a `Boolean`, this will be a function that always returns `true` or always returns `false`—clearly not very interesting for testing the behavior of our function.

EXERCISE 8.19

Hard: We want to generate a function that *uses its argument* in some way to select which `Int` to return. Can you think of a good way of expressing this? This is a very open-ended and challenging design exercise. See what you can discover about this problem and if there's a nice general solution that you can incorporate into the library we've developed so far.

EXERCISE 8.20

You're strongly encouraged to venture out and try using the library we've developed! See what else you can test with it, and see if you discover any new idioms for its use or

[12] Recall that in chapter 7 we introduced the idea of *free theorems* and discussed how parametricity frees us from having to inspect the behavior of a function for every type of argument. Still, there are many situations where being able to generate functions for testing is useful.